

COMP0496 – Programação A

Programação Orientada a Objetos — Parte 3

Herança, Polimorfismo e Composição

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

O Problema — Código Duplicado

Herança Simples

Sobrescrita de Métodos

Polimorfismo e Duck Typing

Composição — A Alternativa

Herança Múltipla e MRO

Classes Abstratas

Prática — Folha de Pagamento

Resumo

O Problema — Código Duplicado

No food truck **O Podrão**, temos três lanches:

```
1 class XTudo:
2     def __init__(self, preco):
3         self.preco = preco
4         self.nome = "X-Tudo"
5
6     def descrever(self):
7         return f"{self.nome}:
8             R${self.preco:.2f}"
9
10    def preparar(self):
11        print("Fritando hambú
12            rguer...")
```

```
1 class Dogao:
2     def __init__(self, preco):
3         self.preco = preco
4         self.nome = "Dogão"
5
6     def descrever(self):
7         return f"{self.nome}:
8             R${self.preco:.2f}"
9
10    def preparar(self):
11        print("Montando hot-
12            dog...")
```

```
1 class Acai:
2     def __init__(self, preco):
3         self.preco = preco
4         self.nome = "Açaí"
5
6     def descrever(self):
7         return f"{self.nome}:
8             R${self.preco:.2f}"
9
10    def preparar(self):
11        print("Batendo açaí...")
```

Violação do Princípio DRY

Don't Repeat Yourself — código duplicado é fonte de bugs.

- `__init__` e `descrever` são **idênticos** nas três classes
- Se precisar adicionar `calorias`: alterar **três** classes
- Se corrigir um bug em `descrever`: corrigir em **três** lugares
- E se o cardápio crescer para 15 itens?

Solução

Extrair o que é **comum** para uma classe base $\Rightarrow$ **Herança**.

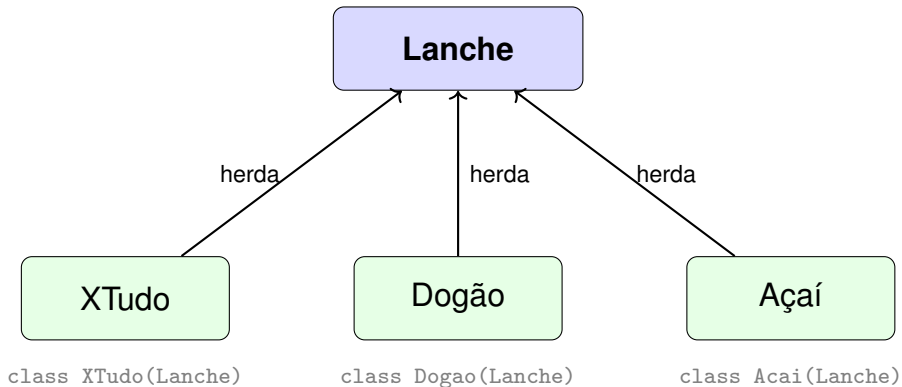
Herança Simples

Herança modela o relacionamento “é um” (*is-a*):

Afirmação	Verdade?	Herança?
XTudo é um Lanche	✓	Sim
Dogão é um Lanche	✓	Sim
Açaí é um Lanche	✓	Sim
Pedido é uma CaixaRegistradora	×	Não
Pedido <i>tem</i> Lanches	✓	Composição

Regra de Ouro

Se “B é um A” faz sentido no domínio $\Rightarrow$ `class B(A)`.




```
1 class Lanche:
2     def __init__(self, nome, preco):
3         self.nome = nome
4         self.preco = preco
5
6     def descrever(self):
7         return f"{self.nome}: R${self.preco:.2f}"
8
9     def preparar(self):
10        print(f"Preparando {self.nome}...")
```

O que mudou?

Todo código comum agora vive em **um único lugar**.

```
1 class XTudo(Lanche):
2     def __init__(self, preco, molho="especial"):
3         super().__init__("X-Tudo", preco)
4         self.molho = molho
5
6     def preparar(self):
7         print(f"Fritando hambúrguer com molho {self.molho}...")
```

```
1 class Dogao(Lanche):
2     def __init__(self, preco, tamanho="grande"):
3         super().__init__("Dogão", preco)
4         self.tamanho = tamanho
5
6     def preparar(self):
7         print(f"Montando hot-dog {self.tamanho}...")
```

```
1 x = XTudo(25.0)
2 d = Dogao(18.0)
3
4 # descrever() é herdado de Lanche -- funciona sem reescrever!
5 print(x.descrever()) # X-Tudo: R$25.00
6 print(d.descrever()) # Dogão: R$18.00
7
8 # preparar() foi sobrescrito em cada subclasse
9 x.preparar() # Fritando hambúrguer com molho especial...
10 d.preparar() # Montando hot-dog grande...
```

Regra

Se a subclasse **não** redefine um método, usa o da superclasse.

```
1 x = XTudo(25.0)
2
3 isinstance(x, XTudo) # True
4 isinstance(x, Lanche) # True (XTudo É UM Lanche)
5 isinstance(x, Dogao) # False
6
7 type(x) == XTudo # True
8 type(x) == Lanche # False (type() é exato)
```

Verificação	<code>isinstance</code>	<code>type() ==</code>
Considera herança?	Sim	Não
Uso recomendado	Geral	Raro

Sobrescrita de Métodos

Sobrescrita (*override*): a subclasse fornece sua própria versão de um método herdado.

```
1 class Acai(Lanche):
2     def __init__(self, preco, complementos=None):
3         super().__init__("Açaí", preco)
4         self.complementos = complementos or ["granola", "banana"]
5
6     def preparar(self):
7         # Sobrescreve completamente o método de Lanche
8         print(f"Batendo açaí com {', '.join(self.complementos)}...")
```

Atenção

O método da subclasse **substitui** o da superclasse — não são acumulados.

Quando queremos **estender** (não substituir) o comportamento do pai:

```
1 class XTudoEspecial(XTudo):
2     def __init__(self, preco, extras=None):
3         super().__init__(preco, molho="da casa")
4         self.extras = extras or ["bacon", "ovo"]
5
6     def descrever(self):
7         base = super().descrever() # chama Lanche.descrever()
8         return f"{base} + {' , '.join(self.extras)}"
```

```
1 xe = XTudoEspecial(32.0)
2 print(xe.descrever())
3 # X-Tudo: R$32.00 + bacon, ovo
```

```
1 class Coxinha(Lanche):
2     def __init__(self, preco):
3         # ESQUECEU super().__init__() !!!
4         self.recheio = "frango"
5
6 c = Coxinha(8.0)
7 c.recheio # "frango" --OK
8 c.descrever() # AttributeError: 'Coxinha' has no attribute 'nome'
```

Por quê?

Sem `super().__init__()`, os atributos `nome` e `preco` nunca são criados.

Correção

Sempre chame `super().__init__()` no início do `__init__` da subclasse.

Polimorfismo e Duck Typing

Polimorfismo: objetos de tipos diferentes respondem à mesma mensagem, cada um à sua maneira.

```
1 pedido = [XTudo(25.0), Dogao(18.0), Acai(15.0)]
2
3 for item in pedido:
4     item.preparar() # cada classe executa seu próprio preparar()
```

Saída:

```
Fritando hambúrguer com molho especial...
Montando hot-dog grande...
Batendo açaí com granola, banana...
```

Essência

O código **não precisa saber** o tipo exato — basta que o objeto tenha o método.

“If it walks like a duck and quacks like a duck, then it must be a duck.”

```
1 class Bebida: # NÃO herda de Lanche!
2     def __init__(self, nome, preco):
3         self.nome = nome
4         self.preco = preco
5
6     def preparar(self):
7         print(f"Servindo {self.nome}...")
8
9     def descrever(self):
10        return f"{self.nome}: R${self.preco:.2f}"
```

```
1 pedido = [XTudo(25.0), Dogao(18.0), Bebida("Guaraná", 6.0)]
2
3 for item in pedido:
4     print(item.descrever())
5     item.preparar()
```

Saída:

```
X-Tudo: R$25.00
Fritando hambúrguer com molho especial...
Dogão: R$18.00
Montando hot-dog grande...
Guaraná: R$6.00
Servindo Guaraná...
```

Observação

Bebida não herda de Lanche, mas funciona no mesmo loop porque tem os mesmos métodos. Isso é **duck typing**.

Composição — A Alternativa

Relacionamento	Mecanismo	Exemplo
“É um” (<i>is-a</i>)	Herança	XTudo é um Lanche
“Tem um” (<i>has-a</i>)	Composição	Pedido tem Lanches
“Tem um” (<i>has-a</i>)	Composição	Pedido tem FormaPagamento
“É um” (<i>is-a</i>)	Herança	Pix é uma FormaPagamento

```
1 class FormaPagamento:
2     def processar(self, valor):
3         raise NotImplementedError
4
5 class Pix(FormaPagamento):
6     def processar(self, valor):
7         print(f"Pix de R${valor:.2f} confirmado!")
8
9 class Pedido:
10     def __init__(self, pagamento):
11         self.itens = [] # composição: tem lanches
12         self.pagamento = pagamento # composição: tem forma de pgto
13
14     def adicionar(self, item):
15         self.itens.append(item)
16
17     def fechar(self):
18         total = sum(i.preco for i in self.itens)
19         self.pagamento.processar(total)
```

```
1 pix = Pix()
2 pedido = Pedido(pix)
3 pedido.adicionar(XTudo(25.0))
4 pedido.adicionar(Bebida("Guaraná", 6.0))
5 pedido.fechar()
6 # Pix de R$31.00 confirmado!
```

Gang of Four

“Favor composition over inheritance.”

Se você só quer **reusar** comportamento, sem relação conceitual “é um”, use composição.

Critério	Herança	Composição
Relação “é um” genuína	✓	
Apenas reusar código		✓
Trocar comportamento em runtime		✓
Hierarquia natural e estável	✓	
Flexibilidade máxima		✓

Regra Prática

Comece com composição. Use herança apenas quando o “é um” for claro e estável.

Herança Múltipla e MRO

Python permite herdar de **mais de uma classe**:

```
1 class Promocional:
2     def aplicar_desconto(self, pct):
3         self.preco *= (1 - pct / 100)
4
5 class XTudoPromo(XTudo, Promocional):
6     pass
7
8 xp = XTudoPromo(25.0)
9 xp.aplicar_desconto(10)
10 print(xp.preco) # 22.5
```

Quando há conflito, Python usa o **C3 Linearization** para decidir qual método chamar:

```
1 print(XTudoPromo.__mro__)  
2 # (XTudoPromo, XTudo, Lanche, Promocional, object)
```

Python busca o método na ordem da MRO: da esquerda para a direita.

Conselho

Herança múltipla pode causar confusão. Na prática:

- Use **mixins** simples (como Promocional)
- Prefira **composição** para combinações complexas
- Evite hierarquias diamante

Classes Abstratas

O Problema: Métodos “Esquecidos”



```
1 class Funcionario:
2     def __init__(self, nome, salario_base):
3         self.nome = nome
4         self.salario_base = salario_base
5
6     def calcular_salario(self):
7         raise NotImplementedError("Subclasse deve implementar")
```

```
1 class Atendente(Funcionario):
2     pass # esqueceu de implementar calcular_salario!
3
4 a = Atendente("Maria", 1500)
5 a.calcular_salario() # NotImplementedError --mas só descobre AQUI!
```

Problema

O erro só aparece quando `calcular_salario()` é chamado — pode ser tarde demais.

```
1 from abc import ABC, abstractmethod
2
3 class Funcionario(ABC):
4     def __init__(self, nome, salario_base):
5         self.nome = nome
6         self.salario_base = salario_base
7
8     @abstractmethod
9     def calcular_salario(self):
10         """Cada tipo de funcionário calcula diferente."""
11         pass
```

```
1 # Tentar instanciar a classe abstrata:
2 f = Funcionario("João", 2000)
3 # TypeError: Can't instantiate abstract class Funcionario
4 # with abstract method calcular_salario
```

Vantagem

O erro ocorre na **instanciação**, não na chamada do método — falha cedo!

```
1 class Atendente(Funcionario):
2     def calcular_salario(self):
3         return self.salario_base
4
5 class Cozinheiro(Funcionario):
6     def __init__(self, nome, salario_base, adicional_insalubridade):
7         super().__init__(nome, salario_base)
8         self.adicional = adicional_insalubridade
9
10    def calcular_salario(self):
11        return self.salario_base + self.adicional
12
13 class Entregador(Funcionario):
14     def __init__(self, nome, salario_base, entregas):
15         super().__init__(nome, salario_base)
16         self.entregas = entregas
17
18    def calcular_salario(self):
19        return self.salario_base + self.entregas * 5
```


Aspecto	<code>raise NotImplementedError</code>	ABC + <code>@abstractmethod</code>
Quando dá erro	Na chamada do método	Na instanciação
Documentação	Implícita	Explícita
Forçar implementação	Não	Sim

Princípio

Classes abstratas definem um **contrato**: “toda subclasse deve ter estes métodos”.

Prática — Folha de Pagamento

O Podrão precisa calcular a folha de pagamento mensal.

Cargo	Cálculo	Exemplo
Atendente	Salário base	R\$ 1.500
Cozinheiro	Base + insalubridade	R\$ 1.800 + R\$ 300
Entregador	Base + R\$ 5/entrega	R\$ 1.200 + 80 $\times$ R\$ 5

```
1 equipe = [  
2     Atendente("Maria", 1500),  
3     Cozinheiro("João", 1800, 300),  
4     Entregador("Carlos", 1200, 80),  
5     Atendente("Ana", 1500),  
6 ]  
7  
8 total_folha = 0  
9 for func in equipe:  
10     salario = func.calcular_salario()  
11     total_folha += salario  
12     print(f"{func.nome:.<20} R${salario:>8.2f}")  
13  
14 print(f"'TOTAL':.<20} R${total_folha:>8.2f}")
```

Saída:

```
Maria..... R$ 1500.00  
João..... R$ 2100.00  
Carlos..... R$ 1600.00  
Ana..... R$ 1500.00  
TOTAL..... R$ 6700.00
```

Observe o loop:

```
1 for func in equipe:
2     salario = func.calcular_salario()
```

- O loop **não sabe** se `func` é Atendente, Cozinheiro ou Entregador
- Cada objeto “sabe” calcular seu próprio salário
- Adicionar novo cargo (ex: Gerente) não altera o loop
- Isso é o **Open/Closed Principle**: aberto para extensão, fechado para modificação

Reflexão

Polimorfismo + classes abstratas = código extensível e seguro.

Resumo

Conceito	Palavra-chave	Quando usar
Herança simples	<code>class B(A)</code>	Relação “é um” clara
<code>super()</code>	<code>super().__init__()</code>	Delegar ao construtor pai
Sobrescrita	Override	Especializar comportamento
Polimorfismo	Mesma interface	Tratar objetos uniformemente
Duck typing	(implícito)	Flexibilidade sem herança
Composição	Atributo de instância	Relação “tem um”
Herança múltipla	<code>class C(A,B)</code>	Mixins simples
Classes abstratas	<code>ABC, @abstractmethod</code>	Forçar contrato

1. **DRY**: não duplique código — extraia para a superclasse
2. **Liskov**: subclasse deve ser usável onde a superclasse é esperada
3. **Composição sobre herança**: use herança apenas para “é um” genuíno
4. **Falhe cedo**: prefira ABC a `NotImplementedError`
5. **Polimorfismo**: programe para a interface, não para a implementação

Próxima Aula (Aula 4)

@dataclass, princípios SOLID e introdução a Design Patterns.